Politecnico di Torino

Integrated Systems Architecture

A.Y. 2025/2026

Laboratory 3

Design and implementation of a hardware accelerator for multichannel 1D convolution - Part 2

Delivery instructions

A template for the homework report is available on the *Portale della Didattica*. Students are required to follow the **provided structure** and ensure that the report is **self-contained**, **complete**, **clear**, and **accurate**. **Diagrams**, **graphs**, and **figures** that facilitate the understanding of the design must be included in the report.

Each project prepared for this course must be submitted by uploading it to the *Portale della Didattica*, section *Elaborati*. Only one member of the group must submit the deliverable.

The deliverable must consist solely of a `.pdf` **report**, describing both part 1 and part 2 of the laboratory. The `.pdf` must be called `isa<nn>_lab3.pdf`, where `<nn>` represents the group number (for example, group 8 should name the file `isa08_lab3.pdf`).

Delivery deadline: February 13 at 23:59

1 Introduction

The aim of this second part of the laboratory session is to implement on *Croc*[1] the previously designed hardware accelerator. All the necessary files and scripts are provided in a repository via Github Classroom: `https://classroom.github.com/a/qt3QhPmm`.

Croc is a System-on-Chip developed by ETH Zurich for educational purposes. It is part of the PULP project[2], a collection of IPs described in the SystemVerilog hardware description language (HDL), along with the corresponding simulation and synthesis scripts and the runtime software written in C and RISC-V assembly that enables the use of the platform. The PULP platform is open-source and free and is meant to be used by other research institutes, universities, and companies. You can download it, use it directly or modify it as you see fit. PULP is released under the Solderpad Hardware License v 0.51, which allows commercial use and does not impose any constraints on additional IPs that you develop and connect to the platform.

2 Open-source design flow

During this lab, a fully open-source design flow will be employed. In particular, for the mcu, all the tools introduced in Part 1 will be used, along with the following additional tools:

- **Riscv-gnu-toolchain**[3], a compiler toolchain used to build *bare-metal* programs for RISC-V architectures;
- **Yosys**[4], an open-source logic synthesis suite capable of translating RTL designs into gate-level netlists;
- **OpenROAD**[5], an automated physical design toolchain that performs floorplanning, placement, routing, and optimization of digital integrated circuits;
- **KLayout**[6], a GDSII viewer and editor used to inspect, analyze, and modify the final layout of the integrated circuit (P&R).

This complete open-source **RTL-to-GDSII** flow makes it possible to generate a GDSII (Graphic Data System II) file, which is the de facto industry standard for representing and exchanging integrated circuit (IC) layout data in Electronic Design Automation (EDA). After the final verification steps, the GDSII file can be sent to a foundry for fabrication.

2.1 Setup Workspace

As in Part 1, the same Docker image will be used to provide the workspace. Since the work will be carried out in a different repository, the **Docker container** must be recreated. To do so:

- delete the container (but not the image) used in Part 1 (it can always be relaunched by returning to the previous repository). This can be done either by clicking the delete icon in the **Containers** section of the Docker Desktop application, or via the terminal with `docker rm -f <container_name>`. Typically, `<container_name>` is `iic-osic-tools_xvnc`, but it may differ depending on the operating system. Check Docker Desktop to verify the exact name;
- create a new container by running, inside the new repository, the script corresponding to your operating system;
- you can now start working again through your browser as in Part 1, but using the new repository.

[1] `https://github.com/pulp-platform/croc`
[2] `https://pulp-platform.org/index.html`
[3] `https://github.com/riscv-collab/riscv-gnu-toolchain`
[4] `https://github.com/YosysHQ/yosys`
[5] `https://github.com/the-openroad-project`
[6] `https://www.klayout.de`

2.2 Makefile

You can now verify if you correctly changed repository by printing the new Makefile help message:

```
make help
```

You should obtain an output similar to the following:

```
Croc MCU Makefile
Software Build and RTL Simulation:
  gen-data            Generate input data for simulation using APP_PARAMS
  sw                  Build the selected bare-metal software (PROG=<name>)
  verilator           Build the RTL simulation model and run the testbench
  waves               Open waveform dump with GTKWave

RTL-to-GDSII Flow:
  yosys-flist         Generate croc.flist using Bender
  yosys               Run RTL -> gate-level synthesis using Yosys
  openroad            Perform floorplanning, placement, CTS, routing and signoff
  klayout             Convert the final DEF into a GDS layout using KLayout

Utilities:
  bender-update       Update vendored IPs
  clean               Remove build files and temporary outputs
  clean-all           Remove all auto-generated files (use with caution)
  help                Display this help message

Examples:
  # Build the bare-metal software:
  make sw PROG=example

  # Custom data generation:
  make gen-data APP_PARAMS="--in_len 16 --in_ch 8 --k_len 5 --k_num 4"
```

2.3 Warnings

You may encounter the same issues as in the previous part. Refer to the previous PDF if that happens.

3 RISC-V build & simulation toolchain

3.1 HW accelerator implementation

The first task of this second part is to implement the accelerator within the **mcu**. As shown in Figure 1, the **croc** SoC is composed of two blocks: the **croc domain** and the **user domain**. The **croc domain** acts as the computational backbone; it provides a stable and unmodifiable foundation that must not be altered. The **user domain**, on the other hand, is a flexible module capable of communicating with the **croc domain**. All custom components that a user wishes to add, such as hardware accelerators, are implemented there. An example counter has already been provided and should be used as a reference for implementing the convolution accelerator.

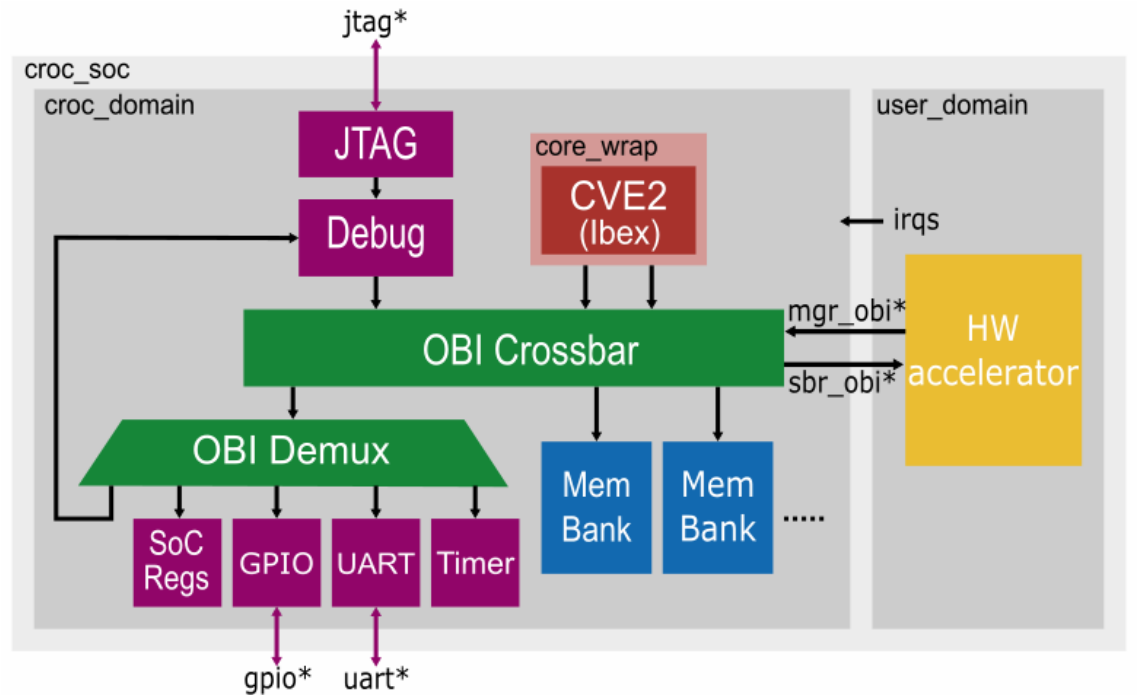

Figure 1: Croc architecture with custom accelerator

You should begin the implementation by vendoring your accelerator into the appropriate folder. This is done by modifying the `Bender.yml` file and then running `make bender-update`.

In the `Bender.yml` file, there are two key sections to examine: the *dependencies*, which specify the other Bender packages on which the main project relies, and the *vendor packages*, which indicate the external packages that must be vendored. Both sections need to be modified in order to integrate the accelerator. As you can observe, the vendor package for the simple counter is already defined and should be used as a reference while creating the vendor package for `conv1d`. **Pay close attention to the paths you specify and to the names of the directories (refer to those already defined in the *dependencies* section).**

Once the `Bender.yml` file has been modified, you can run the command `make bender-update`, which vendors the external packages and updates the dependency graph. If everything runs smoothly, your accelerator will be imported into the `rtl/user_domain` folder.

Suggestion: Since the acclerator repository is imported through a private GitHub link, Bender will request a username and password during the vendoring process (the password corresponds to a GitHub-generated token). To avoid entering your credentials at every vendoring step, you can include the token directly before the GitHub link as follows:

```
https://<token>@github.com/<username>/<reponame>
```

The *conv1d module* is already included in `rtl/user_domain.sv`, but it is currently commented out. Once your accelerator has been correctly imported, you may uncomment it.

3.2 Run firmware

After implementing the accelerator, you must write the firmware that interacts with the hardware module. Before writing your own firmware, you can test how the flow works by compiling the

`hello_world` application using:

```
make sw PROG=example
```

which uses the Riscv-gnu-toolchain to compile the firmware.

Since `example` is the default `PROG`, you can simply run:

```
make sw
```

Once the application has been built, you can run it with Verilator by executing:

```
make verilator
```

If the program finishes with return value 1 and you read `[UART] Hello World!`, the execution completed successfully.

Please note that the `sw` target exposes several parameters that allow you to configure different compiler options. By simply changing the value of the `PROG` variable, you can build different applications. In this project, you may use:

```
make sw PROG=cnt
```

which runs the firmware written to drive the example counter, and

```
make sw PROG=conv1d
```

which runs the firmware that you will write for the accelerator.

The firmware for the accelerator is provided in `sw/applications/conv1d/main.c`. As shown in the code, the `main` function calls two routines:

- `conv1d_sw()`, which performs the convolution in software and is already implemented;
- `conv1d_hw()`, which uses the accelerator to execute the convolution. This function is declared but must be defined inside `conv1d.c`, located in the same directory. For further details, refer to `ASSIGNMENT.md` in the same folder.

The `conv1d_hw()` function requires drivers that control the accelerator's status and control registers. These drivers must be implemented in `sw/drivers/conv1d`. This folder also contains an `ASSIGNMENT.md` file providing additional information.

Once the firmware has been correctly implemented, run it and compare the performance of the hardware and software implementations of the convolution.

Suggestion: Since the software implementation takes significantly longer than the hardware one, you may keep it commented out until the hardware acceleration is working correctly. Once the hardware implementation has been validated, you can uncomment the software version to perform the performance comparison.

Assignment

1. **Integrate** the accelerator into croc.
2. **Write** drivers & firmware and run it.
3. **Compare the performance** of the accelerator with the software implementation.

4 RTL-to-GDSII flow

4.1 Synthesis

The flow that transforms the designed chip into the GDSII format now begins. The first step is synthesis, which requires a filelist that can be generated using Bender with the command:

```
make yosys-flist
```

After generating the filelist, synthesis can be executed by running:

```
make yosys
```

This step performs RTL elaboration, optimization, and technology mapping, producing a gate-level netlist. It validates the synthesizability of the design and prepares it for the place-and-route stage.

Read and try to understand the synthesis reports and the generated logs. In particular, **extract and report the chip area**.

4.2 Place and Route

To perform the full place-and-route flow using the OpenROAD toolchain, run:

```
make openroad
```

This command executes the scripts contained in the `openroad` folder, which describe all the steps of the P&R process. Note that this procedure may take hours.

Read and analyze the **final report**. In particular, you must extract and report the **chip area**, **power consumption**, and **critical path delay**, as well as include the **final layout**.

4.2.1 P&R Improvement (OPTIONAL - EASY)

As can be observed in the post–P&R layout, a significant portion of the chip area remains unused. If the chip were fabricated in this state, this unused space would lead to unnecessary cost, as the resulting die would be larger than required. Try to reduce the chip area by modifying the scripts that define the P&R flow, located in the `openroad/scripts` directory. In particular, you may experiment with adjusting:

- the chip size and placement density in `chip.tcl`;
- the placement and arrangement of macros (i.e., the SRAM blocks) in `floorplan.tcl`;
- the power grid configuration in `power_grid.tcl`.

Attention: If you modify the chip dimensions in `chip.tcl`, you must ensure that they remain consistent with those defined in `padring.tcl`.

The chip area can be reduced by at least half. However, note that reducing the die size increases routing congestion, which in turn significantly raises the runtime of the P&R flow - potentially up to several hours.

Report again the **chip area**, **power consumption**, and **critical path delay**, and include the **updated layout**.

4.3 KLayout (OPTIONAL - CHALLENGING)

After the P&R step, you can run:

```
make klayout
```

to convert the `croc.def` file generated by OpenROAD into `croc_chip.gds`. After a series of verification steps, this GDS file can be sent to a foundry for chip fabrication.

These verification steps include:

- **Design Rule Check** (DRC), which verifies that the layout obeys all manufacturing rules imposed by the target technology (such as spacing, width, and enclosure constraints);
- **Layout Versus Schematic** (LVS), which checks that the physical layout corresponds exactly to the logical netlist, ensuring that no connectivity errors were introduced during the physical design process.

Usually, these steps are performed using the proprietary Siemens Calibre toolchain, but they can also be carried out with KLayout. Since KLayout is still under active development, performing these checks may be more challenging.

The DRC can be executed through the KLayout GUI by selecting *Tools* → *DRC* and loading the file `/foss/pdks/ihp-sg13g2/libs.tech/klayout/tech/drc/sg13g2_maximal.lydrc`. The LVS can be run by selecting *SG13G2 PDK* → *Run KLayout LVS*. Note that LVS requires a `.cdl` file. Try to research how to generate this file in order to perform LVS. Once you have performed both verification steps, try to correct any error you find to obtain a layout that is ready for fabrication.

Assignment

1. **Synthesize** the system with the accelerator using the provided technology.
2. **Report the area** of the synthesis.
3. **Perform** Place&Route.
4. **Report the area, power and timing** results of the system post P&R.
5. (OPTIONAL) **Improve** the P&R.
6. (OPTIONAL) **Perform DRC** and **LVS**